

[util] Installation et imports	
DESCRIPTION	: Bindings Python pour libcups – contrôle le serveur CUPS (impression)
POURQUOI	: Imprimer des fichiers, gérer les imprimantes et les travaux d'impression.
CONTEXTE	: Linux, Raspberry Pi, serveurs d'impression, automatisation, IoT.
INSTALLATION :	
pip install pycups	Bindings Python (nécessite libcups)
PRÉREQUIS SYSTÈME :	
sudo apt install libcups2-dev python3-dev	Ubuntu/Debian – bibliothèques de dev
sudo apt install cups	Serveur CUPS
sudo systemctl start cups	Démarrer CUPS
sudo systemctl enable cups	Activer au démarrage
# Interface web CUPS : http://localhost:631	
IMPORT :	
import cups	Module principal
— FONCTIONS GLOBALES DU MODULE —	
cups.setServer(hostname)	Serveur CUPS cible (défaut: localhost)
cups.setServer("localhost")	Connexion locale
cups.setServer("192.168.1.100")	Serveur CUPS distant
cups.getServer()	Retourne le serveur actuel (str)
cups.setUser(username)	Utilisateur pour les opérations CUPS
cups.setUser("pi")	Utilisateur "pi"
cups.getUser()	Retourne l'utilisateur actuel (str)
cups.setPort(port)	Port CUPS (défaut: 631)
cups.getPort()	Retourne le port actuel (int)
cups.setEncryption(encryption)	Niveau de chiffrement
cups.setEncryption(cups.HTTP_ENCRYPT_IF_REQUESTED)	
cups.setEncryption(cups.HTTP_ENCRYPT_REQUIRED)	
cups.setEncryption(cups.HTTP_ENCRYPT_ALWAYS)	
cups.setEncryption(cups.HTTP_ENCRYPT_NEVER)	
cups.getEncryption()	Retourne le niveau actuel
cups.setPasswordCB(callable)	Callback pour fournir le mot de passe
# callable(prompt) doit retourner le mot de passe (str)	
cups.ppdFindAttr(ppd, name, spec)	Cherche un attribut PPD
cups.ppdFindOption(group, name)	Cherche une option PPD dans un groupe
cups.ppdIsMarked(ppd, option, choice)	Vérifie si une option PPD est cochée
cups.ppdMarkDefaults(ppd)	Marque les options par défaut
cups.ppdMarkOption(ppd, option, choice)	Marque une option PPD
cups.ppdPageSize(ppd, name)	Retourne la taille d'une page nommée
— CONSTANTES HTTP / IPP —	
cups.HTTP_OK	200 – succès
cups.HTTP_CREATED	201 – créé
cups.HTTP_NO_CONTENT	204 – pas de contenu
cups.HTTP_NOT_MODIFIED	304 – non modifié
cups.HTTP_BAD_REQUEST	400 – requête invalide
cups.HTTP_FORBIDDEN	403 – interdit
cups.HTTP_NOT_FOUND	404 – non trouvé
cups.HTTP_SERVER_ERROR	500 – erreur serveur
cups.IPP_OK	Succès IPP
cups.IPP_NOT_FOUND	Ressource IPP non trouvée
cups.IPP_FORBIDDEN	Accès interdit IPP
— VARIABLE D'ENVIRONNEMENT —	
PYCUPS_DEBUG=1	Active les logs de débogage pycups

```
[conn] Connexion – cups.Connection
```

DESCRIPTION : Objet de connexion au serveur CUPS – point d'entrée principal
POURQUOI : Toutes les opérations (impression, gestion, jobs) passent par Connection.
CONTEXTE : Connexion locale ou distante au daemon CUPS.

CONSTRUCTEUR :

```
conn = cups.Connection()          Connexion au serveur par défaut
conn = cups.Connection(host="localhost")  Hôte explicite
conn = cups.Connection(host="192.168.1.10") Serveur CUPS distant
conn = cups.Connection(host="localhost", port=631) Avec port
conn = cups.Connection(
    host="localhost",
    port=631,
    encryption=cups.HTTP_ENCRYPT_IF_REQUESTED
)
# Les valeurs par défaut sont celles de cups.getServer(), getPort(), getEncryption()
# Chaque instanciación appelle httpConnectEncrypt() en interne
```

EXEMPLES :

```
import cups

cups.setUser("monuser")
conn = cups.Connection()
print(conn)                # <cups.Connection object>

# Connexion à un serveur CUPS distant
conn_distant = cups.Connection(host="192.168.1.50", port=631)
```

```
[print] Impression – printFile / printFiles / printTestPage
```

DESCRIPTION : Envoie des fichiers à une imprimante via CUPS
POURQUOI : Automatiser l'impression de PDFs, images, textes depuis Python.
CONTEXTE : Génération de rapports, étiquettes, reçus, photos, documents.

FONCTIONS PRINCIPALES :

```
conn.printFile(printer, filename, title, options) → job_id (int)
conn.printFiles(printer, filenames, title, options) → job_id (int)
conn.printTestPage(name) → job_id (int)
```

— PRINTFILE —

```
conn.printFile(
    printer,          Nom de l'imprimante (str)
    filename,         Chemin absolu du fichier (str)
    title,            Titre du travail (str)
    options           Options d'impression (dict)
)
# Retourne l'ID du travail (int)
# Lève cups.IPPError en cas d'erreur

conn.printFile("HP_LaserJet", "/tmp/doc.pdf", "Mon PDF", {})
conn.printFile("Imprimante", "/home/pi/image.jpg", "Photo", {"fit-to-page": "True"})
conn.printFile("Canon", "/tmp/rapport.pdf", "Rapport", {"copies": "2"})
```

— PRINTFILES (plusieurs fichiers en un seul travail) —

```
conn.printFiles(
    printer,          Nom de l'imprimante
    filenames,        Liste de chemins de fichiers
    title,            Titre du travail
    options           Options (dict)
)
conn.printFiles("HP", ["/tmp/p1.pdf", "/tmp/p2.pdf"], "Job multi", {})
# Tous les fichiers partagent les mêmes options
# Pour des options différentes par fichier → appels printFile séparés
```

— PRINTTESTPAGE —

```
conn.printTestPage("HP_LaserJet")    Imprime la page de test CUPS
# Retourne le job_id
```

— OPTIONS D'IMPRESSION (dict – toutes les valeurs sont des str) —

```
# COPIES ET MISE EN PAGE
{"copies": "2"}                Nombre de copies
{"number-up": "2"}            Pages par feuille (2, 4, 6, 9, 16)
```

<pre>{ "number-up-layout": "lrtb" } { "orientation-requested": "3" } { "page-ranges": "1-3" } { "page-set": "odd" } { "outputorder": "reverse" }</pre>	<pre>Disposition (lrtb, lrbt, rltb, rlbt, tblr, tblr, btlr, btrl) Portrait=3, Paysage=4, Portrait inversé=5, Paysage inversé=6 Pages à imprimer (ex: "1-3", "1,3,5") "all", "even", "odd" "normal" ou "reverse"</pre>
<pre># FORMAT DE PAGE { "media": "A4" } { "media": "Letter" } { "fit-to-page": "True" } { "scaling": "75" } { "natural-scaling": "75" }</pre>	<pre>Format papier : A4, Letter, Legal, A3... Adapter à la page Mise à l'échelle en % (1-100) Mise à l'échelle naturelle</pre>
<pre># MARGES (en points, 1 point = 1/72 pouce) { "page-top": "36" } { "page-bottom": "36" } { "page-left": "36" } { "page-right": "36" }</pre>	<pre>Marge supérieure Marge inférieure Marge gauche Marge droite</pre>
<pre># QUALITÉ ET COULEUR { "print-quality": "3" } { "ColorModel": "Gray" } { "blackplot": "True" }</pre>	<pre>Brouillon=3, Normal=4, Élevée=5 "RGB" ou "Gray" Forcer en noir et blanc</pre>
<pre># DUPLEX (recto-verso) { "Duplex": "None" } { "Duplex": "DuplexNoTumble" } { "Duplex": "DuplexTumble" }</pre>	<pre>Recto simple Recto-verso, reliure long bord Recto-verso, reliure court bord</pre>
<pre># TEXTE { "cpi": "12" } { "lpi": "8" } { "wrap": "true" }</pre>	<pre>Caractères par pouce Lignes par pouce Retour à la ligne automatique</pre>
<pre># FINITION { "StapleLocation": "SinglePortrait" } { "Punch": "2Hole" }</pre>	<pre>Agrafage (si supporté) Perforation</pre>
<pre># PRIORITÉ ET TIMING { "job-priority": "50" } { "job-hold-until": "indefinite" } { "job-hold-until": "2024-12-25T09:00:00Z" }</pre>	<pre>Priorité (1-100, 50=normal) "no-hold", "indefinite", "day-time", "evening", "night", "second-shift", "third-shift", "weekend" Date/heure spécifique ISO 8601</pre>
<pre># DIVERS { "raw": "True" } { "prettyprint": "True" } { "ppi": "150" }</pre>	<pre>Passer les données brutes (bypass filtres) Formatage amélioré du texte brut Pixels par pouce (images)</pre>

EXEMPLES :

```
import cups

conn      = cups.Connection()
printers = conn.getPrinters()
nom_imp   = list(printers.keys())[0]      # Première imprimante

# PDF simple
job_id = conn.printFile(nom_imp, "/tmp/document.pdf", "Mon rapport", {})
print(f"Travail {job_id} soumis")

# 2 copies, A4, recto-verso
job_id = conn.printFile(nom_imp, "/tmp/doc.pdf", "Rapport", {
    "copies": "2",
    "media": "A4",
    "Duplex": "DuplexNoTumble",
})

# Image adaptée à la page, 1 copie
conn.printFile(nom_imp, "/tmp/photo.jpg", "Photo", {"fit-to-page": "True"})

# Texte avec mise en forme
conn.printFile(nom_imp, "/tmp/rapport.txt", "Texte", {
    "cpi": "12",
    "lpi": "8",
    "prettyprint": "True",
    "page-left": "30",
```

```
})

# 4 pages par feuille en paysage
conn.printFile(nom_imp, "/tmp/slides.pdf", "Slides", {
    "number-up": "4",
    "orientation-requested": "4",
})
```

[prn] Gestion des imprimantes – getPrinters / addPrinter / deletePrinter... |

DESCRIPTION : Interroge, ajoute, modifie et supprime des imprimantes dans CUPS
POURQUOI : Administrer le parc d'imprimantes depuis Python.
CONTEXTE : Scripts de déploiement, serveurs d'impression, kiosques.

```
— LISTE ET SÉLECTION —
conn.getPrinters()           → dict {nom: {attributs}}
conn.getClasses()           → dict {nom_classe: {attributs}}
conn.getDests()             → dict des destinations disponibles
conn.getDefault()           → str | None – nom de l'imprimante par défaut
conn.setDefault(name)       Définit l'imprimante par défaut
conn.getPrinterAttributes(name) → dict {attribut: valeur} pour une imprimante
conn.getPrinterAttributes(name="HP_Laser") Par nom
conn.getPrinterAttributes(uri="ipp://...") Par URI
conn.getPrinterAttributes(name, requested_attributes=["printer-state"]) Attributs filtrés
```

```
— ATTRIBUTS COURANTS RETOURNÉS PAR getPrinters() —
printers["NomImp"]["printer-info"]      Nom lisible humain (str)
printers["NomImp"]["printer-location"]   Localisation (str)
printers["NomImp"]["device-uri"]         URI du périphérique (str)
printers["NomImp"]["printer-state"]      État: 3=inactive, 4=impression, 5=arrêté
printers["NomImp"]["printer-state-message"] Message d'état (str)
printers["NomImp"]["printer-is-shared"]  Partagé (bool)
printers["NomImp"]["printer-is-accepting-jobs"] Accepte des travaux (bool)
printers["NomImp"]["queued-job-count"]   Nombre de travaux en file (int)
printers["NomImp"]["printer-make-and-model"] Marque et modèle (str)
printers["NomImp"]["printer-type"]       Type CUPS (int, bitmask)
printers["NomImp"]["printer-uri-supported"] URI IPP (str)
```

```
— ÉTATS D'IMPRIMANTE —
3      IPP_PRINTER_IDLE – inactive
4      IPP_PRINTER_PROCESSING – en cours
5      IPP_PRINTER_STOPPED – arrêtée
```

```
— AJOUT / MODIFICATION —
conn.addPrinter(
    name,                Nom de la file d'attente
    filename=None,       Chemin local du fichier PPD
    ppdname=None,        Nom PPD depuis getPPDs()
    ppd=None,            Objet cups.PPD (un seul des 3 ppd)
    info=None,           Description lisible humain
    location=None,       Localisation
    device=None          URI du périphérique
)
```

NB : filename, ppdname et ppd sont mutuellement exclusifs

```
conn.addPrinter("Mon_Imprimante",
    device="usb://HP/LaserJet?serial=ABC123",
    ppdname="HP/hp-laserjet_5-ps.ppd.gz",
    info="HP LaserJet 5 Bureau",
    location="Salle de réunion"
)
conn.addPrinter("PDF_Virtuelle",
    device="cups-pdf:/",
    ppdname="CUPS-PDF/CUPS-PDF_opt.ppd"
)
conn.addPrinter("Réseau",
    device="ipp://192.168.1.100/printers/HP",
    ppdname="HP/hp-laserjet_6mp-ps.ppd.gz"
)
```

```
— ENABLE / DISABLE / ACCEPT / REJECT —
conn.enablePrinter(name)      Active l'imprimante (traite les jobs)
conn.disablePrinter(name)     Désactive l'imprimante
```

conn.acceptJobs(name)	Accepte les nouveaux travaux
conn.rejectJobs(name)	Refuse les nouveaux travaux
— SUPPRESSION —	
conn.deletePrinter(name)	Supprime une imprimante
conn.deleteClass(class_name)	Supprime une classe d'imprimantes
— CONFIGURATION —	
conn.setPrinterInfo(name, info)	Modifie la description
conn.setPrinterLocation(name, location)	Modifie la localisation
conn.setPrinterDevice(name, device_uri)	Modifie l'URI du périphérique
conn.setPrinterShared(name, shared)	Partage (True/False)
conn.setPrinterOpPolicy(name, policy)	Politique d'opération
conn.setPrinterErrorPolicy(name, policy)	Politique d'erreur
conn.setPrinterJobSheets(name, start, end)	Feuilles de séparation
conn.setPrinterUsersAllowed(name, allowed)	Liste blanche d'utilisateurs
conn.setPrinterUsersDenied(name, denied)	Liste noire d'utilisateurs
— OPTIONS PAR DÉFAUT —	
conn.addPrinterOptionDefault(name, option, value)	Ajoute une option par défaut réseau
conn.deletePrinterOptionDefault(name, option)	Supprime une option par défaut
# Exemple : forcer le duplex par défaut	
conn.addPrinterOptionDefault("HP_Laser", "Duplex", "DuplexNoTumble")	
— CLASSES —	
conn.addPrinterToClass(name, class_name)	Ajoute une imprimante à une classe
conn.deletePrinterFromClass(name, class_name)	Retire une imprimante d'une classe

```
EXEMPLES :
import cups

conn = cups.Connection()

# Lister toutes les imprimantes
printers = conn.getPrinters()
for nom, attrs in printers.items():
    etat = attrs["printer-state"]
    etat_str = {3: "Inactive", 4: "Impression", 5: "Arrêtée"}.get(etat, "Inconnu")
    print(f"{nom} – {attrs['device-uri']} – {etat_str}")

# Imprimante par défaut
default = conn.getDefault()
print(f"Imprimante par défaut : {default}")

# Ajouter une imprimante réseau
conn.addPrinter("Bureau_HP",
    device="ipp://192.168.1.50/printers/LaserJet",
    ppdname="HP/hp-laserjet_p2055-ps.ppd.gz",
    info="HP LaserJet P2055 Bureau",
    location="Open Space"
)
conn.enablePrinter("Bureau_HP")
conn.acceptJobs("Bureau_HP")
conn.setDefault("Bureau_HP")
```

[job]	Gestion des travaux – getJobs / cancelJob / moveJob...	
-------	--	--

DESCRIPTION : Liste, annule, déplace et inspecte les travaux d'impression
POURQUOI : Surveiller et contrôler la file d'impression depuis Python.
CONTEXTE : Monitoring, annulation automatique, redirection de travaux.

— LISTE DES TRAVAUX —	
conn.getJobs()	Travaux en cours (par défaut)
conn.getJobs(which_jobs="not-completed", my_jobs=False, limit=-1, first_job_id=-1, requested_attributes=None)	"not-completed" "completed" "all" True = seulement les travaux de l'user Nombre max (-1 = pas de limite) ID de départ (-1 = depuis le début) Liste d'attributs à retourner
→ dict {job_id (int): {attributs (dict)}}	

— ATTRIBUTS COURANTS D'UN TRAVAIL —	
-------------------------------------	--

```

job["job-name"]           Titre du travail
job["job-state"]          État (int, voir constantes ci-dessous)
job["job-state-reasons"]  Raison de l'état (list)
job["job-printer-uri"]    URI de l'imprimante
job["job-originating-user-name"] Utilisateur soumettant le travail
job["job-k-octets"]       Taille approximative en Ko
job["time-at-creation"]   Timestamp de création (int)
job["time-at-processing"] Timestamp début traitement (int)
job["time-at-completed"]  Timestamp de fin (int)
job["copies"]             Nombre de copies
job["media"]              Format papier

```

— ÉTATS D'UN TRAVAIL (job-state) —

```

3           Pending – en attente
4           Pending-Held – en attente retenu
5           Processing – en cours d'impression
6           Processing-Stopped – traitement stoppé
7           Canceled – annulé
8           Aborted – abandonné
9           Completed – terminé

```

— ATTRIBUTS D'UN TRAVAIL PAR ID —

```

conn.getJobAttributes(jobid)      Tous les attributs d'un travail
conn.getJobAttributes(jobid, requested_attributes=["job-state", "job-name"])

```

— ANNULATION ET PURGE —

```

conn.cancelJob(jobid)             Annule un travail
conn.cancelJob(jobid, purge_job=False) Sans purger les données
conn.cancelJob(jobid, purge_job=True)  Purge les données du travail
conn.cancelAllJobs(uri)             Annule tous les travaux d'une imprimante
conn.cancelAllJobs(uri, my_jobs=True) Seulement les travaux de l'utilisateur
conn.cancelAllJobs(uri, purge_jobs=True) Avec purge des données

```

— DÉPLACEMENT ET REDIRECTION —

```

conn.moveJob(job_id=42, job_printer_uri="ipp://localhost/printers/Autre")
conn.moveJob(printer_uri="ipp://localhost/printers/HP", job_printer_uri="ipp://...")
# printer_uri OU job_id (pas les deux obligatoires)

```

— RETENIR / LIBÉRER —

```

conn.setJobHoldUntil(jobid, "indefinite") Retient le travail indéfiniment
conn.setJobHoldUntil(jobid, "no-hold")    Libère le travail
conn.setJobHoldUntil(jobid, "day-time")    Imprime en journée
conn.setJobHoldUntil(jobid, "night")       Imprime la nuit

```

— REDÉMARRAGE —

```

conn.restartJob(job_id)             Redémarre un travail terminé
conn.restartJob(job_id, job_hold_until="no-hold")

```

— AUTHENTIFICATION —

```

conn.authenticateJob(jobid)          Authentifie un travail retenu
conn.authenticateJob(jobid, auth_info=["user", "password"])

```

— DOCUMENT D'UN TRAVAIL —

```

conn.getDocument(printer_uri, job_id, document_number)
# Retourne dict avec le fichier temporaire

```

EXEMPLES :

```

import cups

conn = cups.Connection()

# Lister tous les travaux en cours
jobs = conn.getJobs()
for jid, attrs in jobs.items():
    print(f"Job {jid}: {attrs.get('job-name')} – état {attrs.get('job-state')}")

# Annuler tous les travaux d'une imprimante
imp_uri = "ipp://localhost/printers/HP_LaserJet"
conn.cancelAllJobs(imp_uri, purge_jobs=True)

# Imprimer et surveiller
jid = conn.printFile("HP_LaserJet", "/tmp/doc.pdf", "Test", {})
import time
while True:
    attrs = conn.getJobAttributes(jid)

```

```
etat = attrs["job-state"]
if etat >= 7:
    # 7=annulé, 8=abandonné, 9=terminé
    print(f"Travail {jid} terminé avec état {etat}")
    break
time.sleep(1)
```

[ppd] PPD – PostScript Printer Description – getPPD / getPPDs / cups.PPD

DESCRIPTION : Accède aux fichiers PPD qui décrivent les capacités des imprimantes
POURQUOI : Lire les options disponibles, configurer les imprimantes finement.
CONTEXTE : Administration d'imprimantes, sélection dynamique d'options.

— OBTENIR LE PPD D'UNE IMPRIMANTE —

conn.getPPD(name)

→ str – chemin du fichier PPD temporaire

conn.getPPD3(name)

→ (status, modtime, filename)

conn.getPPD3(name, modtime=t, filename=f)

Récupère si plus récent que modtime

conn.getServerPPD(ppd_name)

→ str – PPD serveur dans un fichier tmp

— LISTE DES PPD DISPONIBLES —

conn.getPPDs()

→ dict {ppd_name: {attributs}}

conn.getPPDs(

limit=0,

→ Nombre max de résultats (0=illimité)

exclude_schemes=None,

Schémas à exclure (liste)

include_schemes=None,

Schémas à inclure (liste)

ppd_natural_language=None,

Langue (ex: "fr")

ppd_device_id=None,

ID périphérique IEEE 1284

ppd_make=None,

Fabricant (ex: "HP")

ppd_make_and_model=None,

Marque + modèle

ppd_model_number=-1,

Numéro de modèle

ppd_product=None,

Produit PostScript

ppd_psversion=None,

Version PostScript

ppd_type=None,

Type de PPD

)

Attributs courants des entrées :

ppds["nom"]["ppd-make"]

Fabricant

ppds["nom"]["ppd-make-and-model"]

Marque + modèle

ppds["nom"]["ppd-natural-language"]

Langue

ppds["nom"]["ppd-device-id"]

ID IEEE 1284

— OBJET cups.PPD —

ppd_file = conn.getPPD("NomImprimante")

Fichier PPD temporaire

ppd = cups.PPD(ppd_file)

Charge et parse le PPD

ppd.attributes

Liste des attributs PPD

ppd.constraints

Contraintes entre options

ppd.groups

Groupes d'options

ppd.markDefaults()

Marque les options par défaut

ppd.markOption(option, choice)

Marque une option

ppd.nondefaultsMarked()

Vrai si des options non-défaut sont marquées

```
EXEMPLES :
import cups

conn = cups.Connection()

# Lister les PPD HP disponibles
ppds = conn.getPPDs(ppd_make="HP")
for nom, attrs in list(ppds.items())[:5]:
    print(f"{nom}: {attrs['ppd-make-and-model']}")

# Inspecter le PPD d'une imprimante
ppd_path = conn.getPPD("HP_LaserJet")
ppd = cups.PPD(ppd_path)
for group in ppd.groups:
    print(f"Groupe: {group.name}")
    for option in group.options:
        print(f" Option: {option.keyword} = {option.defchoice}")
        for choice in option.choices:
            print(f"    {choice['choice']}: {choice['text']}")
```

[sub] Abonnements et notifications – createSubscription / getNotifications

DESCRIPTION : S'abonne aux événements CUPS pour recevoir des notifications
POURQUOI : Réagir en temps réel aux changements d'état (jobs, imprimantes).
CONTEXTE : Monitoring, tableaux de bord, alertes d'impression.

FONCTIONS PRINCIPALES :

conn.createSubscription(uri, recipient_uri=None, events=None, lease_duration=-1)	URI de l'imprimante ou "/" URI de notification (ippfanout:, rss:...) Liste d'événements à surveiller Durée en secondes (-1 = défaut)
---	---

→ int – subscription_id

— ÉVÉNEMENTS DISPONIBLES —

"all"	Tous les événements
"printer-state-changed"	Changement d'état de l'imprimante
"job-created"	Nouveau travail créé
"job-completed"	Travail terminé
"job-stopped"	Travail arrêté
"printer-added"	Nouvelle imprimante ajoutée
"printer-deleted"	Imprimante supprimée
"server-started"	Serveur CUPS démarré
"server-stopped"	Serveur CUPS arrêté

conn.getSubscriptions(uri) → liste d'IDs d'abonnements
conn.cancelSubscription(id) Annule un abonnement
conn.renewSubscription(id, lease_duration=-1) Renouvelle un abonnement
conn.getNotifications(subscription_ids) → liste des notifications

EXEMPLES :

```
conn = cups.Connection()  
  
# S'abonner aux événements de tous les jobs  
sub_id = conn.createSubscription("/",  
    events=["job-created", "job-completed", "printer-state-changed"]  
)  
  
# Récupérer les notifications  
notifs = conn.getNotifications([sub_id])  
for n in notifs:  
    print(n)  
  
# Annuler l'abonnement  
conn.cancelSubscription(sub_id)
```

[adm] Administration serveur – adminGetServerSettings / getDevices |

DESCRIPTION : Configure le serveur CUPS et liste les périphériques disponibles
POURQUOI : Administrer le serveur CUPS depuis Python (partage, sécurité...)
CONTEXTE : Scripts de déploiement, configuration automatisée.

— PARAMÈTRES DU SERVEUR —

conn.adminGetServerSettings()	→ dict des paramètres serveur
conn.adminSetServerSettings(settings)	Modifie les paramètres serveur

Constantes pour les clés de paramètres :

cups.CUPS_SERVER_DEBUG_LOGGING	"0" ou "1" – logs de débogage
cups.CUPS_SERVER_REMOTE_ADMIN	"0" ou "1" – admin distante
cups.CUPS_SERVER_REMOTE_PRINTERS	"0" ou "1" – imprimantes distantes
cups.CUPS_SERVER_SHARE_PRINTERS	"0" ou "1" – partage d'imprimantes
cups.CUPS_SERVER_USER_CANCEL_ANY	"0" ou "1" – annulation par utilisateurs

— PÉRIPHÉRIQUES DISPONIBLES —

conn.getDevices()	→ dict {device_uri: {attributs}}
conn.getDevices(limit=0, exclude_schemes=None, include_schemes=None)	Nombre max de périphériques Schémas à exclure (ex: ["dnssd"]) Schémas à inclure (ex: ["usb", "ipp"])

Attributs courants :

devs["uri"]	Classe : "direct", "network"...
devs["uri"]["device-class"]	Info lisible humain
devs["uri"]["device-make-and-model"]	Marque + modèle

FICHIERS DE CONFIGURATION

conn.getFile(resource, filename)R cup re un fichier de config CUPS
conn.getFile("/admin/conf/cupsd.conf", "/tmp/cupsd.conf")
conn.putFile(resource, filename)Envoie un fichier de config CUPS
conn.putFile("/admin/conf/cupsd.conf", "/tmp/cupsd_modifie.conf")

EXEMPLES :

conn = cups.Connection()

Activer le partage d'imprimantes
settings = conn.adminGetServerSettings()
settings[cups.CUPS_SERVER_SHARE_PRINTERS] = "1"
conn.adminSetServerSettings(settings)

Lister les p r ph riques USB connect s
devices = conn.getDevices(include_schemes=["usb"])
for uri, attrs in devices.items():
 print(f"{uri}: {attrs.get('device-make-and-model', 'Inconnu')}")

[util] Exceptions et codes d'erreur

DESCRIPTION : Exceptions lev es par pycups
POURQUOI : G rer proprement les erreurs d'impression et d'administration.
CONTEXTE : Imprimante hors ligne, fichier introuvable, droits insuffisants.

EXCEPTIONS :

cups.IPPError Erreur IPP g n rique
cups.IPPError(status_code, description)
e.args[0] Code de statut IPP (int)
e.args[1] Description de l'erreur (str)

CODES IPP COURANTS :

cups.IPP_OK 0x0000 Succ s
cups.IPP_NOT_FOUND 0x0406 Ressource non trouv e
cups.IPP_FORBIDDEN 0x0403 Op ration non autoris e
cups.HTTP_* pour les erreurs HTTP sous-jacentes

EXEMPLES :

import cups

conn = cups.Connection()
try:
 conn.printFile("ImprimanteInexistante", "/tmp/doc.pdf", "Test", {})
except cups.IPPError as e:
 print(f"Erreur IPP {e.args[0]}: {e.args[1]}")
except RuntimeError as e:
 print(f"Erreur : {e}")

[util] Patterns et recettes

DESCRIPTION : Patterns robustes pour les cas d'usage courants
POURQUOI : Le serveur CUPS peut  tre indisponible ou l'imprimante hors ligne.
CONTEXTE : Production, Raspberry Pi, serveurs d'impression automatis s.

IMPRESSION ROBUSTE AVEC RETRY

import cups, time

def imprimer_robuste(fichier, titre, options={}, tentatives=3):
 for i in range(tentatives):
 try:
 conn = cups.Connection()
 printers = conn.getPrinters()
 if not printers:
 raise RuntimeError("Aucune imprimante disponible")
 nom = conn.getDefault() or list(printers.keys())[0]
 jid = conn.printFile(nom, fichier, titre, options)
 print(f"  Travail {jid} soumis   {nom}")
 return jid
 except cups.IPPError as e:

```

        print(f"Tentative {i+1}/{tentatives}: IPP {e.args[0]} - {e.args[1]}")
        time.sleep(2)
    raise RuntimeError("Échec d'impression après plusieurs tentatives")

— MONITORING DE LA FILE D'IMPRESSION —————
def surveiller_imprimantes():
    conn = cups.Connection()
    printers = conn.getPrinters()
    jobs = conn.getJobs(which_jobs="all")
    for nom, attrs in printers.items():
        etat = attrs["printer-state"]
        nb_j = attrs.get("queued-job-count", 0)
        print(f"{nom}: état={etat}, travaux en attente={nb_j}")
    print(f"Total travaux actifs : {len(jobs)}")

— IMPRIMER UN PDF GÉNÉRÉ EN MÉMOIRE —————
import cups, tempfile, os

def imprimer_pdf_bytes(pdf_bytes, titre, options={}):
    with tempfile.NamedTemporaryFile(suffix=".pdf", delete=False) as f:
        f.write(pdf_bytes)
        tmp_path = f.name
    try:
        conn = cups.Connection()
        nom = conn.getDefault() or list(conn.getPrinters().keys())[0]
        jid = conn.printFile(nom, tmp_path, titre, options)
        return jid
    finally:
        os.unlink(tmp_path)

— VÉRIFIER QU'UNE IMPRIMANTE EST PRÊTE —————
def imprimante_prete(nom):
    conn = cups.Connection()
    attrs = conn.getPrinterAttributes(nom)
    etat = attrs.get("printer-state", 0)
    accepte = attrs.get("printer-is-accepting-jobs", False)
    return etat in (3, 4) and accepte

— ATTENDRE LA FIN D'UN TRAVAIL —————
def attendre_travail(conn, jid, timeout=60):
    debut = time.time()
    while time.time() - debut < timeout:
        try:
            attrs = conn.getJobAttributes(jid)
            etat = attrs["job-state"]
            if etat == 9:
                return "completed"
            elif etat in (7, 8):
                return "failed"
        except cups.IPPError:
            return "not_found"
        time.sleep(1)
    return "timeout"

```

=====

RÉCAPITULATIF DES IMPORTS

=====

import cups	Module principal (pycups)
# Connexion	
cups.setUser("monuser")	Avant d'instancier Connection
cups.setServer("localhost")	
conn = cups.Connection()	
# Impression	
conn.printFile(printer, filename, title, options)	→ job_id
conn.printFiles(printer, filenames, title, options)	→ job_id
conn.printTestPage(name)	→ job_id
# Imprimantes	
conn.getPrinters()	→ {nom: {attrs}}
conn.getDefault()	→ str None
conn.addPrinter(name, device=, ppdname=)	

```

conn.enablePrinter(name) / disablePrinter(name)
conn.acceptJobs(name)      / rejectJobs(name)
conn.deletePrinter(name)

# Travaux
conn.getJobs(which_jobs="not-completed")    → {jid: {attrs}}
conn.getJobAttributes(jid)                  → {attrs}
conn.cancelJob(jid)
conn.cancelAllJobs(uri)
conn.moveJob(job_id=jid, job_printer_uri=uri)

# PPD
conn.getPPD(name)                        → chemin fichier tmp
conn.getPPDs(ppd_make="HP")              → {ppd_name: {attrs}}
ppd = cups.PPD(ppd_path)                 Charge le PPD

# Administration
conn.adminGetServerSettings()             → dict
conn.adminSetServerSettings(settings)
conn.getDevices()                         → {uri: {attrs}}

# Exceptions
cups.IPPError                             e.args[0] = code, e.args[1] = msg

# — GUIDE DE DÉCISION RAPIDE —————
# Imprimer un fichier                     → conn.printFile(...)
# Options d'impression                   → dict passé à printFile (toutes valeurs str)
# Lister imprimantes                     → conn.getPrinters()
# Travaux en cours                       → conn.getJobs()
# Annuler un travail                     → conn.cancelJob(jid)
# Ajouter une imprimante                 → conn.addPrinter(name, device=..., ppdname=...)
# Imprimante par défaut                  → conn.getDefault() / setDefault(name)
# PPD d'une imprimante                   → conn.getPPD(name) puis cups.PPD(path)
# Surveiller les événements              → conn.createSubscription + getNotifications

```

=====